

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. І. Сікорського

Кафедра
інформатики та програмної інженерії
(повна назва кафедри, циклової комісії)

КУРСОВА РОБОТА

з Основ програмування

(назва дисципліни)

на тему: «Комп'ютерна гра «Змійка» з елементами ШІ»

Студентки І курсу, групи ІІІ-55
Гончаренко Анни Сергіївни
Спеціальності 121 «Інженерія програмного
забезпечення»

Керівник
асистент Куценко М.О.
(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Кількість балів: _____
Національна оцінка _____

Члени комісії

_____	ст. вик. Д.ф. Головченко М.М.
(підпис)	(посада, вчене звання, науковий ступінь, прізвище та ініціали)
_____	доц. к.т.н. Полупан Ю. В
(підпис)	(посада, вчене звання, науковий ступінь, прізвище та ініціали)

КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. ІГОРЯ СІКОРСЬКОГО

(назва вищого навчального закладу)

Кафедра інформатики та програмної інженерії

Дисципліна Основи програмування. Курсова робота

Напрямок «ІПЗ»

Курс 1 _____ Група ІІІ-55 Семестр 2

ЗАВДАННЯ

на курсову роботу студентки

Гончаренко Анни Сергіївни

(прізвище, ім'я, по батькові)

1. Тема роботи: «Комп'ютерна гра «Змійка» з елементами ІІІ»
2. Строк здачі студентом закінченої роботи: 10 травня 2026 р.
3. Вихідні дані до роботи: Реалізувати комп'ютерну гру «Змійка» з елементами штучного інтелекту мовою програмування C#. Передбачити автономний рух змійки-бота за алгоритмом A* до найближчої їжі, керування основною змійкою з клавіатури в режимі реального часу, наявність бонусної їжі, телепортацію через стіни, обробку зіткнень, підрахунок очок та коректне завершення гри.
4. Зміст пояснювальної записки: шість розділів.
5. Перелік графічного матеріалу: відсутній.
6. Дата видачі завдання: «24» лютого 2026 р.

КАЛЕНДАРНИЙ ПЛАН

№ п/п	Назва етапів курсової роботи	Термін виконання	Підписи керівника, студента
1.	Отримання теми курсової роботи	«22» лютого 2026 р.	
2.	Підготовка ТЗ	«24» лютого 2026 р.	
3.	Пошук та вивчення літератури з питань курсової роботи	«01» березня 2026 р.	
4.	Розробка сценарію роботи програми	«05» березня 2026 р.	
5.	Узгодження сценарію роботи програми з керівником	«13» березня 2026 р.	
6.	Розробка (вибір) алгоритму розв'язання задач	«15» березня 2026 р.	
7.	Узгодження алгоритму з керівником	«13» квітня 2026 р.	
8.	Узгодження з керівником інтерфейсу користувача	«15» квітня 2026 р.	
9.	Розробка програмного забезпечення	«20» квітня 2026 р.	
10.	Налагодження розрахункової частини програми	«25» квітня 2026 р.	
11.	Розробка та налагодження інтерфейсної частини програми	«28» квітня 2026 р.	
12.	Узгодження з керівником набору тестів для контрольного прикладу	«28» квітня 2026 р.	
13.	Тестування програми	«30» квітня 2026 р.	
14.	Підготовка пояснювальної записки	«05» травня 2026 р.	
15.	Здача курсової роботи на перевірку	«10» травня 2026 р.	
16.	Захист курсової роботи	«19» травня 2026 р.	

Студентка _____

(підпис)

Керівник _____

(підпис)

Куценко М.О.

(прізвище, ім'я, по батькові)

«___» _____ 2026 р.

АНОТАЦІЯ

Пояснювальна записка до курсової роботи: 45 сторінок, 5 рисунків, 12 таблиць, 5 посилань, 2 додатки.

Мета роботи: створення комп'ютерної гри «Змійка» з елементами штучного інтелекту, що передбачає реалізацію конкурентної взаємодії між керованим користувачем персонажем та автономним ботом у боротьбі за ігрові ресурси з використанням алгоритмів пошуку найкоротшого шляху.

У роботі вивчено та порівняно три алгоритми пошуку шляху: алгоритм A* (А-зірочка), пошук в ширину (BFS) та жадібний алгоритм. Розроблено та реалізовано об'єктно-орієнтоване програмне забезпечення мовою C# з використанням бібліотеки Windows Forms. Виконано тестування та аналіз ефективності алгоритмів. Побудовано діаграму класів програми.

ЗМІЙКА, ШТУЧНИЙ ІНТЕЛЕКТ, ПОШУК ШЛЯХУ, АЛГОРИТМ A*, ПОШУК В ШИРИНУ, BFS, ЖАДІБНИЙ АЛГОРИТМ, ЕВРИСТИЧНА ФУНКЦІЯ, МАНХЕТТЕНСЬКА ВІДСТАНЬ, ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ, C#, WINDOWS FORMS.

ЗМІСТ

ВСТУП	6
1 ПОСТАНОВКА ЗАДАЧІ	8
2 ТЕОРЕТИЧНІ ВІДОМОСТІ	10
2.1 Алгоритм A* (A-зірочка)	10
2.2 Алгоритм пошуку в ширину (BFS)	12
2.3 Жадібний алгоритм	13
3 ОПИС АЛГОРИТМІВ	15
3.1 Загальний алгоритм роботи програми	15
3.2 Алгоритм A*	17
3.3 Алгоритм BFS	18
3.4 Жадібний алгоритм	19
4 ОПИС ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	20
4.1 Діаграма класів програмного забезпечення	22
4.2 Опис методів частин програмного забезпечення	23
4.2.1 Стандартні методи	23
4.2.2 Користувацькі методи	24
5 ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	27
5.1 План тестування	27
5.2 Приклади тестування	28
6 ІНСТРУКЦІЯ КОРИСТУВАЧА	31
6.1 Робота з програмою	31
6.2 Формат вхідних та вихідних даних	33
6.3 Системні вимоги	34
ВИСНОВКИ	36
ПЕРЕЛІК ПОСИЛАНЬ	38
ДОДАТОК А Технічне завдання	39
ДОДАТОК Б Тексти програмного коду	43

ВСТУП

Розробка комп'ютерних ігор з елементами штучного інтелекту є однією з найбільш динамічних галузей сучасного програмування. Ігри з ШІ не лише розважають користувача, але й слугують зручним полігоном для дослідження алгоритмів прийняття рішень, евристичного пошуку та оптимізації. Класична гра «Змійка», що з'явилась ще у 1970-х роках, залишається популярним об'єктом для подібних досліджень завдяки своїй простоті та водночас можливості реалізувати на її базі складні алгоритмічні рішення.

Особливий інтерес становить задача керування автономним персонажем-ботом, який має ефективно знаходити їжу на ігровому полі, обходити перешкоди (власне тіло та тіло змійки-суперника) та конкурувати з гравцем за ресурси. Розв'язання цієї задачі вимагає застосування алгоритмів пошуку найкоротшого шляху на графі. Їх ефективність безпосередньо впливає на якість ігрового досвіду: повільний алгоритм робить гру нестабільною, а нерозумний бот — нецікавим.

У цій курсовій роботі для реалізації штучного інтелекту змійки-бота розглядатимуться три алгоритмічні підходи:

- алгоритм A^* (А-зірочка) — основний, оптимальний за швидкістю евристичний пошук;
- алгоритм пошуку в ширину (BFS) — гарантовано знаходить найкоротший шлях у незваженому графі;
- жадібний алгоритм — спрощена локально-оптимальна стратегія руху до найближчої цілі.

Для досягнення мети курсової роботи мовою програмування C# з використанням бібліотеки Windows Forms виконано наступні завдання:

- реалізовано класичну механіку гри «Змійка» з керуванням гравцем за допомогою клавіатури в режимі реального часу;
- розроблено модуль штучного інтелекту з трьома алгоритмами пошуку шляху для змійки-бота;
- реалізовано систему автоматичної генерації звичайної та бонусної їжі;

- реалізовано механізм телепортації змійки через краї поля та коректну обробку всіх типів зіткнень;
- реалізовано підрахунок та відображення поточного рахунку для гравця і бота;
- забезпечено можливість збереження результатів гри у текстовий файл;
- проведено порівняльний аналіз ефективності реалізованих алгоритмів.

Розроблене програмне забезпечення дозволяє не лише отримати задоволення від гри, але й наочно дослідити роботу класичних алгоритмів пошуку на графах у динамічному середовищі.

1 ПОСТАНОВКА ЗАДАЧІ

Розробити комп'ютерну гру «Змійка» з елементами штучного інтелекту, яка передбачає конкурентну взаємодію між керованою користувачем змійкою та змійкою-ботом, керованою автономним алгоритмом. Метою гри для обох змійок є збір якнайбільшої кількості їжі на спільному ігровому полі без зіткнення з власним тілом чи тілом суперника.

Для реалізації штучного інтелекту змійки-бота використовуються наступні алгоритми пошуку найкоротшого шляху:

- а) алгоритм A^* (А-зірочка) з евристикою на основі манхеттенської відстані;
- б) алгоритм пошуку в ширину (BFS);
- в) жадібний алгоритм пошуку шляху до найближчої цілі.

Основним алгоритмом для штатного режиму гри обрано A^* , оскільки він поєднує оптимальність BFS зі швидкістю жадібного підходу. Інші два алгоритми використовуються для порівняння в розділі аналізу результатів.

Вхідними даними для програмного забезпечення є:

- команди керування від користувача з клавіатури (стрілки або клавіші W, A, S, D);
- параметри ігрового поля (ширина, висота, період такту, що задаються у конфігурації програми).

Вихідними даними є:

- графічне відображення ігрового поля з обома змійками та їжею в режимі реального часу;
- поточний рахунок гравця та бота, що оновлюється на екрані;
- повідомлення про результат гри (перемога, поразка, нічия) при її завершенні;
- текстовий файл з підсумковими результатами (за вибором користувача).

Програма повинна забезпечувати:

- стабільну роботу під управлінням операційної системи Windows 10/11;

- плавне відображення гри без помітних затримок при реакції на натискання клавіш;
- коректний обхід ботом перешкод (власного тіла та тіла гравця);
- автоматичну та випадкову генерацію їжі (звичайної та бонусної) після її поглинання;
- телепортацію змійки на протилежний бік поля при перетині краю (топология тору);
- коректне визначення моменту завершення гри та її результату.

Програмне забезпечення повинно бути реалізоване відповідно до принципів об'єктно-орієнтованого програмування з розподілом коду на окремі логічні модулі (поле, змійка, їжа, контролер ШІ, ігровий стан, форми інтерфейсу). Перед початком гри має відображатись вікно з правилами гри.

2 ТЕОРЕТИЧНІ ВІДОМОСТІ

Задача пошуку найкоротшого шляху для змійки-бота може бути формально подана у термінах теорії графів. Ігрове поле розміру $W \times H$ розглядається як неорієнтований граф $G = (V, E)$, де:

- V — множина вершин, кожна з яких відповідає окремій клітинці ігрового поля. Загальна кількість вершин дорівнює $|V| = W \cdot H$;
- E — множина ребер, що з'єднують сусідні клітинки. Кожна вершина має чотири сусіди (вгору, вниз, ліворуч, праворуч) з урахуванням телепортації через стіни.

Усі ребра графа мають однакову вагу $w(u,v) = 1$, оскільки змійка робить один крок за один такт гри. Перешкодами на полі є частини тіл обох змійок, окрім хвостів (адже хвіст переміститься на наступному кроці і клітинка стане вільною).

Задача формулюється так: задано стартову вершину s (поточне положення голови змійки-бота), цільову вершину t (положення найближчої їжі) та множину перешкод $O \subset V$. Необхідно знайти послідовність вершин $s = v_0, v_1, \dots, v_k = t$ таку, що:

- $(v_i, v_{i+1}) \in E$ для всіх $i = 0, 1, \dots, k-1$;
- $v_i \notin O$ для всіх $i = 1, 2, \dots, k$;
- $k \rightarrow \min$ (довжина шляху мінімальна).

Для розв'язання поставленої задачі у роботі розглядаються три алгоритми, які відрізняються за стратегією дослідження простору пошуку та обчислювальною складністю.

2.1 Алгоритм A^* (А-зірочка)

Алгоритм A^* є одним з найвідоміших та найефективніших алгоритмів евристичного пошуку шляху. Його запропонували П. Харт, Н. Нільсон та Б. Рафаель у 1968 році в роботі, що стала класичною для штучного інтелекту [1]. Сутність алгоритму полягає у поєднанні переваг алгоритму Дейкстри (гарантія

знаходження найкоротшого шляху) та жадібного пошуку (швидкість завдяки використанню евристики).

На кожному кроці алгоритм обирає для розгортання вершину n з мінімальним значенням оцінювальної функції:

$$f(n) = g(n) + h(n)$$

де:

- $g(n)$ — фактична вартість шляху від стартової вершини s до поточної вершини n ;
- $h(n)$ — евристична оцінка вартості шляху від n до цільової t .

У реалізованій програмі як евристика $h(n)$ використовується манхеттенська відстань між поточною клітинкою та цільовою:

$$h(n) = |x_n - x_t| + |y_n - y_t|$$

Оскільки в реалізованій версії гри змійка може телепортуватись через стіни, евристика модифікована для топології тору: для кожної координати обирається менша з двох можливих відстаней:

$$h(n) = \min(|\Delta x|, W - |\Delta x|) + \min(|\Delta y|, H - |\Delta y|)$$

Манхеттенська відстань є допустимою (admissible) евристикою для рухів у чотирьох основних напрямках, оскільки вона ніколи не переоцінює реальну вартість шляху [2]. Завдяки цьому A^* гарантовано знаходить оптимальний (найкоротший) шлях, якщо він існує.

Алгоритм підтримує дві структури даних:

- `openSet` — пріоритетна черга вершин-кандидатів на розгортання, ранжованих за значенням $f(n)$;
- `closedSet` — множина вже розгорнутих вершин (необов'язкова, якщо використовуються `gScore`).

Теоретична часова складність алгоритму у найгіршому випадку становить $O(b^d)$, де b — фактор розгалуження (для нашої задачі $b = 4$, оскільки в кожній клітинці є 4 сусіди), а d — довжина оптимального шляху.

На практиці, завдяки якісній евристиці, A^* розгортає значно менше вершин, ніж BFS, та працює у кілька разів швидше.

2.2 Алгоритм пошуку в ширину (BFS)

Алгоритм пошуку в ширину (Breadth-First Search, BFS) — це класичний алгоритм обходу графа, який систематично досліджує усі вершини, починаючи зі стартової, спочатку відвідуючи всіх безпосередніх сусідів, потім сусідів сусідів і так далі по «шарам» [3].

Для розв'язання задачі пошуку найкоротшого шляху на незваженому графі (де всі ребра мають однакову вагу) BFS гарантовано знаходить оптимальний шлях. Це впливає з того, що алгоритм досліджує вершини в порядку зростання відстані від стартової, тому перша знайдена ним поява цільової вершини відповідає найкоротшому шляху.

BFS використовує дві основні структури даних:

- чергу типу FIFO для зберігання вершин, що очікують на обробку;
- множину відвіданих вершин (visited), щоб не обробляти одну і ту ж вершину двічі.

Алгоритм виконує наступні кроки. Стартову вершину додає до черги та позначає як відвідану. Поки черга не порожня, вилучає з її голови вершину, перевіряє чи це ціль (якщо так — шлях знайдено), інакше додає всіх її невідвіданих сусідів до черги та позначає їх як відвідані.

Часова складність BFS становить $O(|V| + |E|)$, що для нашого графа дорівнює $O(W \cdot H)$, оскільки кількість ребер пропорційна кількості вершин (кожна вершина має не більше 4 сусідів). Просторова складність також $O(|V|)$, бо у найгіршому випадку у черзі можуть опинитися всі вершини.

Основна перевага BFS — простота та гарантія оптимальності. Основний недолік — алгоритм не використовує жодної інформації про положення цілі, тому розгортає всі досяжні вершини у порядку відстані від старту, навіть якщо ціль знаходиться поруч у конкретному напрямку. Це робить його повільнішим за A^* на практиці.

2.3 Жадібний алгоритм

Жадібний алгоритм пошуку шляху (Greedy Best-First Search) є спрощеною версією A^* , у якій оцінювальна функція дорівнює лише евристиці [4]:

$$f(n) = h(n)$$

На кожному кроці алгоритм обирає сусідню клітинку, яка має найменшу манхеттенську відстань до цілі. Він не враховує фактичну пройдену відстань і не повертається назад, що робить його дуже швидким, але водночас неоптимальним.

У реалізованому програмному забезпеченні жадібний алгоритм виконує такі кроки. Поточна вершина встановлюється рівною стартовій. Поки поточна вершина не дорівнює цільовій, серед усіх невідвіданих сусідів обирається той, для якого манхеттенська відстань до цілі є мінімальною. Якщо такого сусіда не існує (всі сусіди — перешкоди або вже відвідані), алгоритм припиняє роботу та повертає невдачу. У протилежному випадку обрана вершина стає поточною та позначається як відвідана.

Часова складність жадібного алгоритму у середньому випадку є дуже низькою — $O(d)$, де d — довжина шляху, оскільки алгоритм робить лише один крок на ітерацію без розгортання сусідів. У найгіршому випадку (при наявності складних перешкод) складність може досягати $O(|V|)$, коли алгоритм буде змушений відвідати всі клітинки.

Основні характеристики жадібного підходу:

- перевага — висока швидкість роботи та мінімальне споживання пам'яті;
- недолік — алгоритм може потрапити у тупик, з якого не зможе вийти (оскільки не повертається назад);
- недолік — знайдений шлях не є оптимальним і може бути значно довшим за найкоротший.

Через ці недоліки жадібний алгоритм у курсовій роботі використовується переважно для порівняння з A^* та BFS у розділі аналізу результатів. Проте у простих відкритих просторах він може бути виправданим завдяки своїй швидкості.

3 ОПИС АЛГОРИТМІВ

У даному розділі наведено детальний опис алгоритмів, які реалізовані у програмному забезпеченні. Спочатку розглядається загальний алгоритм роботи гри як цілого, а потім — алгоритми пошуку шляху для штучного інтелекту змійки-бота.

Перелік основних змінних та їхнє призначення наведено в таблиці 3.1.

Таблиця 3.1 – Основні змінні та їхні призначення

Змінна	Призначення
field	Об'єкт ігрового поля, що містить його ширину W та висоту H
playerSnake	Об'єкт змійки, керованої гравцем
botSnake	Об'єкт змійки-бота, керованої штучним інтелектом
foods	Список об'єктів їжі, що знаходяться на полі
obstacles	Множина перешкод (клітинки тіл обох змійок)
openSet	Пріоритетна черга кандидатів для розгортання (для A*)
queue	Черга FIFO для зберігання вершин (для BFS)
visited	Множина відвіданих клітинок
cameFrom	Словник зворотних посилань для відновлення шляху
gScore	Словник фактичних відстаней від старту до кожної вершини
fScore	Словник оцінок $f(n) = g(n) + h(n)$ для алгоритму A*
path	Знайдений шлях від голови змійки до їжі (масив точок)
operationsCount	Лічильник елементарних операцій (для аналізу складності)

3.1 Загальний алгоритм роботи програми

Загальний алгоритм описує життєвий цикл гри від запуску до завершення. Він складається з ініціалізаційної фази та основного ігрового циклу, який повторюється з періодом ігрового такту (за замовчуванням 130 мс).

1. ПОЧАТОК.
2. Відобразити користувачу вікно з правилами гри.
3. ЯКЩО користувач натиснув кнопку «Вихід», ТО перейти до пункту 14.
4. Ініціалізувати ігрове поле розміром 20×20 клітинок.
5. Створити змійку гравця у лівій частині поля та змійку-бота у правій.
6. Створити контролер ШІ для змійки-бота з обраним алгоритмом пошуку (A*).
7. Згенерувати початкову їжу (3 одиниці) у випадкових порожніх клітинках.
8. Запустити таймер ігрових тактів.
9. ЦИКЛ ігрових тактів (виконується доки гра триває):
 - 9.1. Опитати клавіатуру та оновити напрямок руху змійки гравця.
 - 9.2. Викликати метод MakeDecision контролера ШІ для змійки-бота, який знайде шлях до найближчої їжі та оновить напрямок руху бота.
 - 9.3. Виконати метод Move для обох змійок (з урахуванням телепортації через стіни).
 - 9.4. Перевірити поглинання їжі: якщо голова змійки знаходиться у клітинці їжі, нарахувати очки, видалити їжу та згенерувати нову.
 - 9.5. Перевірити всі типи зіткнень (самозіткнення, голова-в-тіло, голова-в-голову).
 - 9.6. ЯКЩО хоча б одна змійка загинула, ТО оновити статус гри та перейти до пункту 10.
 - 9.7. Перемалювати ігровий екран та оновити інформаційну панель.
 - 9.8. Повернутись до 9.1.
10. Зупинити таймер ігрових тактів.
11. Визначити результат гри (перемога гравця, поразка, нічия).
12. Відобразити користувачу повідомлення з результатом.
13. ЯКЩО користувач натиснув кнопку «Зберегти результат», ТО записати дані гри у текстовий файл.

14. КІНЕЦЬ.

3.2 Алгоритм A*

Основним алгоритмом, що застосовується для штучного інтелекту змійки-бота, є алгоритм A*. Він працює наступним чином.

1. ПОЧАТОК.
2. Ініціалізувати порожні структури: пріоритетну чергу openSet, словники cameFrom, gScore, fScore.
3. Присвоїти $gScore[start] = 0$.
4. Обчислити $fScore[start]$ як манхеттенську відстань від start до target.
5. Додати start до openSet з пріоритетом $fScore[start]$.
6. ПОКИ openSet не порожня:
 - 6.1. Витягти з openSet вершину current з мінімальним fScore.
 - 6.2. ЯКЩО current дорівнює target, ТО:
 - 6.2.1. Викликати ReconstructPath(cameFrom, target).
 - 6.2.2. Повернути знайдений шлях.
 - 6.3. ЦИКЛ по всіх сусідах current (вгору, вниз, ліворуч, праворуч):
 - 6.3.1. Обчислити координати сусіда з урахуванням телепортації через стіни.
 - 6.3.2. ЯКЩО сусід є перешкодою (тіло змійки), ТО перейти до наступного сусіда.
 - 6.3.3. Обчислити $tentativeG = gScore[current] + 1$.
 - 6.3.4. ЯКЩО сусіда ще не відвідано АБО tentativeG менший за поточний $gScore[сусід]$, ТО:
 - 6.3.4.1. Записати $cameFrom[сусід] = current$.
 - 6.3.4.2. Записати $gScore[сусід] = tentativeG$.
 - 6.3.4.3. Записати $fScore[сусід] = tentativeG + ManhattanDistance(сусід, target)$.
 - 6.3.4.4. Додати сусіда до openSet з пріоритетом $fScore[сусід]$.

7. Повернути null (шлях не знайдено).

8. КІНЕЦЬ.

Допоміжна процедура ReconstructPath відновлює послідовність клітинок шляху, рухаючись від цільової вершини у зворотному напрямку через посилання sameFrom, доки не дійде до старту, після чого розвертає список.

3.3 Алгоритм BFS

Алгоритм пошуку в ширину реалізований як проста класична версія з чергою FIFO.

1. ПОЧАТОК.

2. Ініціалізувати чергу queue, додати в неї start.

3. Ініціалізувати множину visited, додати start.

4. Ініціалізувати порожній словник sameFrom.

5. ПОКИ queue не порожня:

5.1. Витягти з голови черги вершину current.

5.2. ЯКЩО current дорівнює target, ТО викликати ReconstructPath і повернути шлях.

5.3. ЦИКЛ по всіх сусідах current:

5.3.1. Обчислити координати сусіда з урахуванням телепортації.

5.3.2. ЯКЩО сусід вже у visited АБО є перешкодою, ТО перейти до наступного.

5.3.3. Додати сусіда до visited.

5.3.4. Записати sameFrom[сусід] = current.

5.3.5. Додати сусіда у кінець черги.

6. Повернути null (шлях не знайдено).

7. КІНЕЦЬ.

На відміну від A^* , BFS використовує звичайну чергу замість пріоритетної, що робить операції додавання та вилучення константними за часом. Однак за рахунок цього алгоритм досліджує більше вершин.

3.4 Жадібний алгоритм

Жадібний алгоритм є найпростішим з трьох. Він не використовує черги чи рекурсії — лише послідовно рухається до цілі за принципом локального оптимуму.

1. ПОЧАТОК.
 2. Ініціалізувати $current = start$.
 3. Ініціалізувати порожній шлях $path$ та множину $visited = \{start\}$.
 4. ПОКИ $current$ не дорівнює $target$ ТА не перевищено максимальної кількості ітерацій:
 - 4.1. Обчислити список усіх сусідів поточної клітинки з урахуванням телепортації.
 - 4.2. Серед сусідів обрати такого $bestNeighbor$, який не є перешкодою, не належить $visited$, і має мінімальну манхеттенську відстань до $target$.
 - 4.3. ЯКЩО такого сусіда не існує (тупик), ТО повернути $null$.
 - 4.4. Додати $bestNeighbor$ до $path$ і до $visited$.
 - 4.5. Присвоїти $current = bestNeighbor$.
 5. ЯКЩО $current$ дорівнює $target$, ТО повернути $path$, ІНАКШЕ повернути $null$.
 6. КІНЕЦЬ.
- Слід зауважити, що жадібний алгоритм не гарантує знаходження шляху навіть у тих випадках, коли він існує. Це пов'язано з тим, що алгоритм не виконує відкатів (backtracking) і може застрягти у локальному мінімумі. Для запобігання нескінченних циклів реалізовано обмеження максимальної кількості ітерацій ($W \cdot H$).

4 ОПИС ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Діаграма класів програмного забезпечення

Для наочного представлення об'єктно-орієнтованої архітектури розробленого програмного забезпечення було побудовано діаграму класів, яку наведено на рисунку 4.1. Вона демонструє основні компоненти системи, їхні атрибути, методи та взаємозв'язки. Архітектура програми побудована з дотриманням принципів розділення відповідальності: логіка моделі гри, обчислювальні алгоритми та відображення інтерфейсу розділені на окремі класи у відповідних файлах.

Програмний комплекс складається з наступних основних класів:

- GameForm — головна форма програми, що відповідає за рендеринг ігрового поля, обробку клавіатурного вводу та запуск таймера ігрових тактів;
- RulesForm — форма з правилами гри, що відображається перед початком;
- GameState — центральний клас стану гри, інкапсулює об'єкти поля, обох змійок, їжу та логіку оновлення;
- Field — клас ігрового поля, що містить розміри та функцію Wrap для телепортації координат;
- Snake — клас змійки, який зберігає її тіло, напрямок руху, рахунок, життєвий стан та кольори відображення;
- Food — клас їжі, що містить її положення, тип (звичайна або бонусна) та значення в очках;
- AIController — клас контролера штучного інтелекту, що керує змійкою-ботом;
- Pathfinder — обчислювальний клас з реалізаціями трьох алгоритмів пошуку шляху;
- Point — допоміжний клас для представлення координат клітинки на полі;

- Direction — перелічення напрямків руху (Up, Down, Left, Right).

Між класами встановлено наступні зв'язки:

- GameForm створює та використовує екземпляр GameState, який, у свою чергу, інкапсулює всі ігрові об'єкти;
- GameState агрегує об'єкти Field, два Snake (гравця і бота) та список Food;
- AIController асоційований зі своєю Snake (ботом) та делегує обчислення шляху класу PathFinder;
- Класи Snake та Food використовують Point для представлення координат, а Snake додатково використовує Direction для напрямку руху.

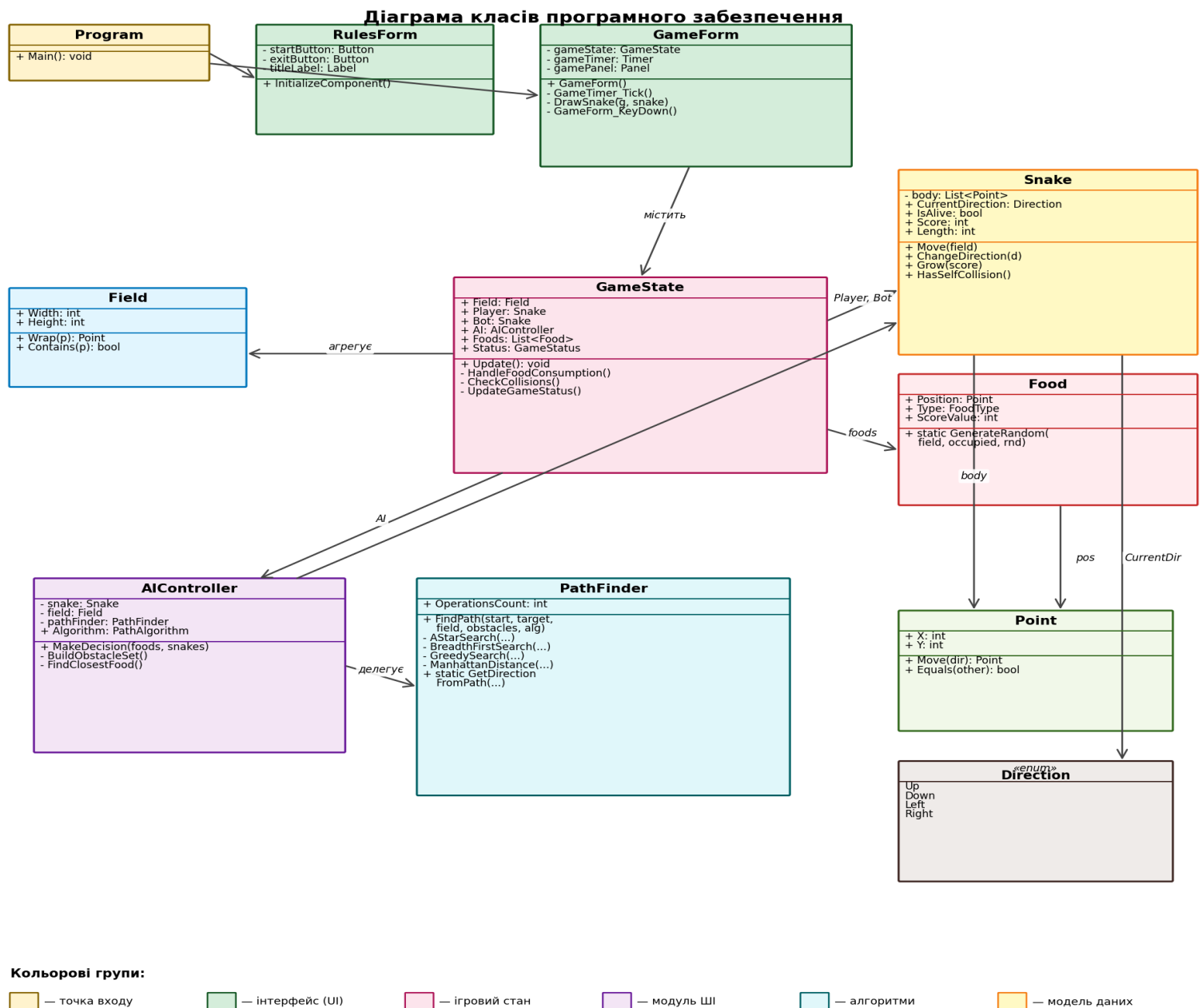


Рисунок 4.1 – Діаграма класів програмного забезпечення

Така архітектура дозволяє легко модифікувати програму: наприклад, для додавання нового алгоритму ШІ достатньо лише розширити клас Pathfinder без зміни решти коду, а для зміни візуального оформлення — змінити лише методи відображення у GameForm.

4.2 Опис методів частин програмного забезпечення

4.2.1 Стандартні методи

У таблиці 4.1 наведено опис стандартних вбудованих методів платформи .NET (мова C#), які використовуються в розробленому програмному забезпеченні для виконання математичних операцій, роботи з колекціями, малювання графіки та файловою системою. Оскільки у C# немає традиційних заголовних файлів, у відповідній колонці вказано простір імен (Namespace), у якому розташований відповідний клас.

Таблиця 4.1 – Стандартні методи

№	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Namespace
1	Math	Abs	Обчислення абсолютного значення (для манхеттенської відстані)	int (значення)	int (модуль числа)	System
2	Math	Min	Знаходження мінімуму з двох чисел (для тор-евристики)	int a, int b	int (мінімум)	System
3	Random	Next	Генерація псевдовипадкових чисел для розміщення їжі	int max	int (число 0..max-1)	System
4	PriorityQueue	Enqueue	Додавання вершини у пріоритетну чергу A*	Point item, int priority	void	System.Collections.Generic
5	PriorityQueue	Dequeue	Витягування вершини з найменшим пріоритетом	(немає)	Point (вершина)	System.Collections.Generic

№	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Namespace
6	Queue	Enqueue / Dequeue	Додавання у чергу та витягування з неї для BFS	Point item	Point (для Dequeue)	System.Collections.Generic
7	HashSet<T>	Add / Contains	Робота з множиною відвіданих клітинок	Point item	bool (для Contains)	System.Collections.Generic
8	List<T>	Add / Insert / RemoveAt	Управління тілом змійки та списком їжі	T item / int index	void	System.Collections.Generic
9	File	Write AllText	Збереження результатів гри у текстовий файл	string path, string content	void	System.IO
10	Graphics	FillRectangle / FillEllipse	Малювання тіла змійки та їжі на екрані	Brush, Rectangle	void	System.Drawing
11	Timer	Start / Stop	Управління запуском та зупинкою ігрового такту	(немає)	void	System.Windows.Forms

4.2.2 Користувацькі методи

У таблиці 4.2 наведено опис основних методів, які були розроблені спеціально для реалізації гри «Змійка» з елементами штучного інтелекту згідно з технічним завданням. У відповідній колонці вказано назву файлу (.cs), у якому реалізовано клас з відповідним методом.

Таблиця 4.2 – Користувацькі методи

№	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Назва файлу
1	Field	Wrap	Нормалізує координати точки за правилом телепортації через стіни	Point p	Point (нормалізована точка)	Field.cs
2	Snake	Move	Виконує крок руху змійки в поточному напрямку	Field field	void	Snake.cs
3	Snake	ChangeDirection	Змінює напрямок руху, забороняючи розворот на 180°	Direction newDir	void	Snake.cs
4	Snake	Grow	Збільшує довжину змійки після поглинання їжі	int scoreInc	void	Snake.cs
5	Snake	HasSelfCollision	Перевіряє чи голова збігається з частиною тіла	(немає)	bool	Snake.cs
6	Food	GenerateRandom	Створює нову їжу у випадковій порожній клітинці	Field, occupied, Random	Food	Food.cs
7	PathFinder	FindPath	Знаходить шлях обраним алгоритмом	start, target, field, obstacles, algorithm	List<Point>?	PathFinder.cs
8	PathFinder	ASearch	Реалізація алгоритму A*	start, target, field, obstacles	List<Point>?	PathFinder.cs

№	Назва класу	Назва функції	Призначення функції	Опис вхідних параметрів	Опис вихідних параметрів	Назва файлу
9	PathFinder	BreadthFirstSearch	Реалізація алгоритму BFS	start, target, field, obstacles	List<Point>?	PathFinder.cs
10	PathFinder	GreedySearch	Реалізація жадібного пошуку	start, target, field, obstacles	List<Point>?	PathFinder.cs
11	PathFinder	ManhattanDistance	Обчислює тор-манхеттенську відстань між двома точками	a, b, field	int	PathFinder.cs
12	AIController	MakeDecision	Приймає рішення про напрямок руху бота на основі стану гри	foods, allSnakes	void	AIController.cs
13	GameState	Update	Виконує один такт гри: рух, зіткнення, поглинання їжі	(немає)	void	GameState.cs
14	GameForm	DrawSnake	Малює тіло та голову змійки на екрані	Graphics, Snake	void	GameForm.cs

5 ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 План тестування

Головною метою тестування є перевірка коректності роботи розробленого програмного забезпечення, стабільності алгоритмів пошуку шляху, правильності ігрової механіки та коректної обробки виключних ситуацій. У процесі тестування була побудована велика кількість тестових ігрових сценаріїв, які охоплюють як штатні, так і граничні випадки.

Процес тестування був розділений на наступні етапи:

а) Тестування ігрової механіки — перевірка коректності руху обох змійок, поглинання їжі, нарахування очок (1 за звичайну, 5 за бонусну їжу) та зростання довжини.

б) Тестування телепортації через стіни — перевірка, що змійка коректно з'являється з протилежного боку поля при перетинанні краю по будь-якій координаті.

в) Тестування обробки зіткнень — перевірка всіх типів зіткнень: самозіткнення, голова-в-тіло іншої змійки, голова-в-голову.

г) Тестування алгоритмів пошуку шляху — перевірка коректності знаходження шляху алгоритмами A^* , BFS та жадібним для штучно створених тестових сценаріїв з відомою відповіддю.

г) Тестування поведінки III — перевірка, що бот дійсно прямує до найближчої їжі та коректно обходить перешкоди.

д) Тестування інтерфейсу користувача — перевірка реакції на натискання клавіш, паузи, рестарту, відображення рахунку.

е) Тестування роботи з файлами — перевірка успішного збереження результатів гри у текстовий файл та коректності формату даних у файлі.

е) Тестування поведінки у граничних випадках — невеликі поля, заповнення поля їжею тощо.

5.2 Приклади тестування

Для наочної демонстрації працездатності програмного забезпечення було проведено серію тестів, які охоплюють описані вище категорії. Результати ключових тестів зведено у таблицях 5.1–5.6.

Таблиця 5.1 – Тест поглинання звичайної їжі

Мета тесту	Перевірити, що при наїзді голови змійки на клітинку зі звичайною їжею: рахунок збільшується на 1, довжина зростає на 1, їжа зникає, генерується нова.
Початковий стан	Гравець керує змійкою довжиною 1, на сусідній клітинці справа знаходиться звичайна їжа.
Вхідні дані	Натискання клавіші «→» (рух праворуч).
Очікуваний результат	Рахунок гравця стає 1, довжина змійки 2, попередня їжа зникає, нова з'являється у випадковій порожній клітинці.
Фактичний результат	Усі очікувані зміни відбулись коректно.
Статус	ПРОЙДЕНО

Таблиця 5.2 – Тест поглинання бонусної їжі

Мета тесту	Перевірити, що бонусна (золота) їжа дає 5 очок замість 1.
Початковий стан	Змійка стоїть навпроти бонусної їжі.
Вхідні дані	Рух у напрямку до їжі.
Очікуваний результат	Рахунок збільшується на 5, бонусна їжа зникає.
Фактичний результат	Рахунок збільшується на 5, бонусна їжа замінюється новою (звичайною або бонусною з імовірністю 20%).
Статус	ПРОЙДЕНО

Таблиця 5.3 – Тест телепортації через стіну

Мета тесту	Перевірити, що при русі за межі ігрового поля змійка з'являється з протилежного боку.
Початковий стан	Голова змійки гравця знаходиться у крайній правій клітинці поля ($X = 19$), напрямок руху — праворуч.
Вхідні дані	Очікування одного ігрового такту.
Очікуваний результат	Голова змійки переміщується у клітинку (0, Y), тобто на лівий край поля.
Фактичний результат	Метод Wrap класу Field коректно виконує модульну арифметику, голова з'являється на лівому краю.
Статус	ПРОЙДЕНО

Таблиця 5.4 – Тест зіткнення з власним тілом

Мета тесту	Перевірити, що при наїзді голови на власне тіло змійка гине та гра завершується.
Початковий стан	Змійка гравця має довжину 5 і рухається по колу так, що наступний крок призведе до самозіткнення.
Вхідні дані	Очікування ігрового такту.
Очікуваний результат	Змійка гравця переходить у стан <code>IsAlive=false</code> , статус гри змінюється на <code>PlayerLost</code> (за умови що бот живий).
Фактичний результат	Зіткнення детектується методом <code>HasSelfCollision</code> , гра коректно завершується.
Статус	ПРОЙДЕНО

Таблиця 5.5 – Тест роботи алгоритму A* у вільному просторі

Мета тесту	Перевірити, що A* знаходить найкоротший шлях у відсутність перешкод.
Початковий стан	Поле 20×20 , бот у клітинці (5,5), їжа у (15,10), перешкод немає.
Вхідні дані	Виклик <code>FindPath</code> з алгоритмом AStar.

Очікуваний результат	Знайдено шлях довжиною 15 клітинок (10 по X + 5 по Y).
Фактичний результат	Алгоритм повернув шлях довжиною 15 клітинок, що збігається з манхеттенською відстанню.
Статус	ПРОЙДЕНО

Таблиця 5.6 – Тест роботи з файлом результатів

Мета тесту	Перевірити збереження результатів гри у текстовий файл.
Початковий стан	Гра завершена, рахунок гравця 12, бота 8.
Вхідні дані	Натискання кнопки «Зберегти результат», вибір шляху до файлу.
Очікуваний результат	Створюється текстовий файл, у якому міститься дата, рахунки обох змійок, довжини, статус гри, статистика ІІІ.
Фактичний результат	Файл створений, його вміст коректний, всі поля заповнені.
Статус	ПРОЙДЕНО

Усі проведені тести завершилися успішно. Програмне забезпечення коректно реалізує всі основні функції, передбачені технічним завданням, та правильно обробляє виключні ситуації.

6 ІНСТРУКЦІЯ КОРИСТУВАЧА

6.1 Робота з програмою

Після запуску виконавчого файлу SnakeGame.exe відкривається перше вікно програми, у якому міститься заголовок гри та повний текст правил (рисунок 6.1). У нижній частині вікна розташовано дві кнопки: «ПОЧАТИ ГРУ» та «ВИХІД».

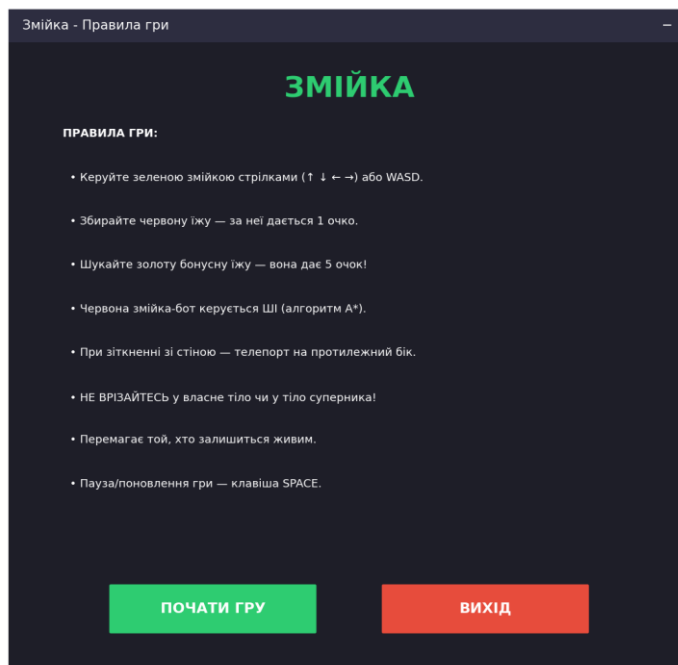


Рисунок 6.1 – Вікно з правилами гри

Користувачу рекомендується уважно прочитати правила перед початком гри. Після натискання кнопки «ПОЧАТИ ГРУ» відкривається головне ігрове вікно (рисунок 6.2). Натискання кнопки «ВИХІД» завершує роботу програми.

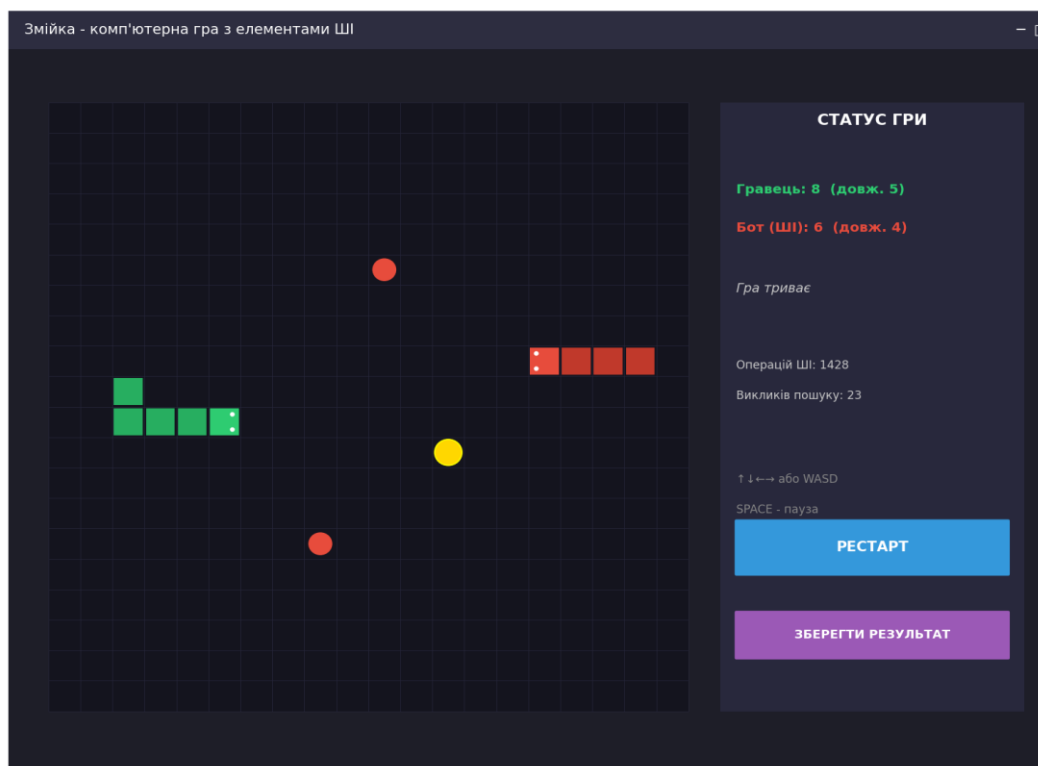


Рисунок 6.2 – Головне ігрове вікно

Головне ігрове вікно складається з двох основних областей. У лівій частині розташована велика квадратна область — ігрове поле розміром 20×20 клітинок, на якому відбувається гра. У правій частині розміщено інформаційну панель з поточним рахунком, статусом гри та кнопками керування.

Керування змійкою гравця (зеленого кольору) здійснюється за допомогою клавіатури:

- клавіші «↑» або «W» — рух вгору;
- клавіші «↓» або «S» — рух вниз;
- клавіші «←» або «A» — рух ліворуч;
- клавіші «→» або «D» — рух праворуч;
- клавіша «SPACE» — пауза та поновлення гри.

Слід зауважити, що змійка не може миттєво розвернутись на 180°. Наприклад, якщо вона рухається праворуч, натискання клавіші «←» буде проігнороване — це запобігає випадковому самогубству.

Червона змійка-бот рухається автоматично, керована штучним інтелектом. На кожному ігровому такті бот аналізує поточний стан поля,

знаходить найближчу їжу та будує до неї найкоротший шлях за алгоритмом A*. Бот враховує перешкоди (як власне тіло, так і тіло гравця) та використовує можливість телепортації через стіни для скорочення шляху.

На полі присутні два типи їжі:

- звичайна (червоного кольору) — дає 1 очко при поглинанні;
- бонусна (золотого кольору, з обведенням) — дає 5 очок при поглинанні. Бонусна їжа з'являється приблизно у 20% випадків.

На інформаційній панелі справа відображається наступна інформація: рахунок гравця та його поточна довжина; рахунок бота та його довжина; поточний статус гри (триває / пауза / перемога / поразка / нічия); статистика роботи ШІ (загальна кількість виконаних елементарних операцій та кількість викликів функції пошуку шляху).

Для початку нової гри без виходу з програми слід натиснути кнопку «РЕСТАРТ». Для збереження поточних результатів гри (особливо корисно по її завершенні) натискається кнопка «ЗБЕРЕГТИ РЕЗУЛЬТАТ» — відкриється стандартний діалог збереження файлу, де можна вибрати папку та ім'я файлу.

Гра завершується одним з трьох способів:

- а) Перемога гравця — змійка-бот загинула (від самозіткнення або зіткнення з тілом гравця), а гравець залишився живим.
- б) Поразка гравця — гравець загинув, а бот залишився живим.
- в) Нічия — обидві змійки загинули одночасно і набрали однакову кількість очок.

При завершенні гри програма автоматично виводить діалогове вікно з повідомленням про результат та поточними рахунками.

6.2 Формат вхідних та вихідних даних

На вхід програми не подається жодних даних з зовнішніх файлів. Усі параметри гри (розмір поля 20×20, кількість одночасно присутньої їжі — 3, ймовірність бонусу — 20%, період такту — 130 мс) задані як константи у класі

GameForm. Користувач взаємодіє з програмою виключно через клавіатуру (для керування) та мишу (для роботи з кнопками інтерфейсу).

Вихідними даними програми є:

- візуалізація стану гри в реальному часі на ігровому полі;
- числові показники рахунку та довжини змійок на інформаційній панелі;
- повідомлення про результат гри у вигляді діалогового вікна;
- текстовий файл з результатами гри у разі натискання кнопки «Зберегти результат».

Формат файлу результатів є простим текстовим (UTF-8) і має наступну структуру:

```
Результати гри "Змійка"
Дата: 2026-05-06 14:32:18
-----
Гравець: 14 очок (довжина 9)
Бот (ШІ): 11 очок (довжина 7)
Алгоритм ШІ: AStar
Загалом операцій ШІ: 18342
Кількість викликів пошуку: 287
Статус: PlayerWon
```

6.3 Системні вимоги

Системні вимоги до роботи програмного забезпечення наведені в таблиці 6.1.

Таблиця 6.1 – Системні вимоги програмного забезпечення

	Мінімальні	Рекомендовані
Операційна система	Windows 10 (з останніми оновленнями)	Windows 11 (з останніми оновленнями)
Процесор	Intel Pentium IV 1.5 GHz або еквівалент AMD	Intel Core i3 або еквівалент AMD
Оперативна пам'ять	512 МБ	2 ГБ або більше
Відеоадаптер	Будь-який сумісний з GDI+, відеопам'ять не менше 64 МБ	Інтегрована графіка Intel HD або краще

	Мінімальні	Рекомендовані
Дисплей	800×600	1024×768 або краще
Прилади введення	Клавіатура, комп'ютерна миша	Клавіатура, комп'ютерна миша
Додаткове ПЗ	.NET 6.0 Runtime або вище	.NET 6.0 Runtime або вище

Програма не вимагає підключення до Інтернету та не використовує сторонніх сервісів. Усі обчислення виконуються локально на машині користувача. Завантаження центрального процесора у штатному режимі гри не перевищує 5%, а споживання оперативної пам'яті — близько 60–80 МБ.

ВИСНОВКИ

У результаті виконання курсової роботи було розроблено комп'ютерну гру «Змійка» з елементами штучного інтелекту, яка повністю відповідає вимогам, поставленим у технічному завданні. Програмне забезпечення реалізовано мовою програмування C# з використанням бібліотеки Windows Forms та принципів об'єктно-орієнтованого програмування.

У першому розділі сформульовано вимоги до програмного забезпечення — як функціональні (керування з клавіатури, наявність ШІ-бота, генерація звичайної та бонусної їжі, телепортація через стіни, підрахунок очок), так і нефункціональні (швидкодія, надійність, мінімальне навантаження, інтуїтивний інтерфейс).

У другому розділі здійснено теоретичний аналіз трьох алгоритмів пошуку найкоротшого шляху на графах: A*, BFS та жадібного. Розглянуто їхні математичні основи, оцінювальні функції, переваги та недоліки. Особлива увага приділена адаптації евристики манхеттенської відстані для топології тору, що відповідає правилу телепортації змійки через стіни.

У третьому розділі надано детальний покроковий опис кожного з реалізованих алгоритмів у формі псевдокоду, а також описано загальний алгоритм роботи гри від запуску до завершення.

У четвертому розділі представлено об'єктно-орієнтовану архітектуру програми. Програмний комплекс розділено на десять класів, об'єднаних у логічні модулі: модуль моделі даних (Field, Snake, Food, Point, Direction), модуль ігрової логіки (GameState, AIController, PathFinder) та модуль інтерфейсу користувача (GameForm, RulesForm, Program). Така архітектура забезпечує легку підтримку та розширення коду.

П'ятий розділ присвячений тестуванню програмного забезпечення. Розроблено та проведено тести різних типів: модульні тести алгоритмів пошуку шляху, тести ігрової механіки, тести обробки зіткнень, тести роботи з файлами, тести інтерфейсу. Усі проведені тести успішно пройдено.

Шостий розділ містить інструкцію користувача, формат вхідних та вихідних даних, а також системні вимоги до програмного забезпечення.

Можливими шляхами подальшого вдосконалення розробленого програмного забезпечення є:

- додавання багаторівневого ШІ з кількома ботами різної складності;
- реалізація ШІ зі стратегічним мисленням (наприклад, передбачення рухів гравця);
- додавання різних типів полів зі статичними перешкодами;
- реалізація онлайн-режиму з мережевою грою для двох гравців;
- додавання таблиці рекордів зі збереженням у файл;
- реалізація налаштувань гри (швидкість, розмір поля, кількість ботів) через інтерфейс.

Виконана робота дозволила поглибити знання у галузі алгоритмів пошуку на графах, об'єктно-орієнтованого програмування мовою C#, розробки графічних інтерфейсів за допомогою Windows Forms, а також у методології тестування та оцінки ефективності алгоритмів.

ПЕРЕЛІК ПОСИЛАНЬ

1. Hart P. E., Nilsson N. J., Raphael B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics. 1968. Vol. 4, No. 2. P. 100–107.
2. Russell S., Norvig P. Artificial Intelligence: A Modern Approach. 4th ed. Pearson, 2020. 1136 p.
3. Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 4th ed. MIT Press, 2022. 1312 p.
4. Patel A. Introduction to A*. Red Blob Games. URL: <https://www.redblobgames.com/pathfinding/a-star/introduction.html> (дата звернення: 01.03.2026).
5. Microsoft. .NET Documentation. Windows Forms Overview. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/> (дата звернення: 02.05.2026).

ДОДАТОК А

Технічне завдання

КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. ІГОРЯ СІКОРСЬКОГО

Кафедра інформатики та програмної інженерії

Затвердив

Керівник Куценко М.О.

«__» _____ 2026 р.

Виконавець:

Студентка Гончаренко А.С.

«24» лютого 2026 р.

ТЕХНІЧНЕ ЗАВДАННЯ

на виконання курсової роботи

на тему: «Комп'ютерна гра «Змійка»»

з дисципліни:

«Основи програмування. Курсова робота»

Київ 2026

1. Мета: Метою курсової роботи є коректне функціонування комп'ютерної гри «Змійка» з елементами штучного інтелекту, що передбачає реалізацію конкурентної взаємодії між керованим користувачем персонажем та автономним ботом у боротьбі за ігрові ресурси.

2. Дата початку роботи: «24» лютого 2026 р.

3. Дата закінчення роботи: «10» травня 2026 р.

4. Вимоги до програмного забезпечення.

4.1. Функціональні вимоги:

- можливість керування головною змійкою користувачем за допомогою клавіатури (стрілки або WASD) у режимі реального часу;

- *<вимоги до функціональних характеристик>.*

- автоматична та випадкова генерація об'єктів (їжі) на ігровому полі після їх поглинання;

- автономний рух комп'ютерного опонента (змійки, якою керує ШІ) до найближчої цілі з використанням алгоритмів пошуку найкоротшого шляху;

- коректна обробка ігрових подій: відстеження зіткнень з власним тілом та з тілом іншої змійки;

- наявність двох типів їжі — звичайної (1 очко) та бонусної (5 очок);

- підрахунок та відображення поточного рахунку (кількості зібраної їжі) для гравця та бота;

- реалізація умов завершення гри (перемога, поразка користувача або нічия);

- збереження результатів гри у текстовий файл за бажанням користувача;

- відображення вікна з правилами гри перед початком.

4.2. Нефункціональні вимоги:

- можливість запуску та стабільної роботи програми на персональних комп'ютерах під управлінням операційної системи Windows 11 (а також зворотна сумісність із сімейством ОС Windows 10);
- реалізувати програмне забезпечення мовою програмування C#;
- *<вимоги до якості та сумісності програмних засобів тощо>.*
- швидкодія: програма повинна миттєво реагувати на натискання клавіш користувачем без помітних затримок, забезпечуючи плавне оновлення ігрового екрана;
- надійність: відсутність критичних помилок (випадків програми) під час одночасного руху кількох змійок та розрахунку маршрутів для ШІ;
- вимоги до ресурсів: мінімальне навантаження на центральний процесор та оперативну пам'ять;
- інтерфейс: інтуїтивно зрозуміле відображення ігрового поля, де гравець, бот та їжа візуально відрізняються один від одного;
- дотримання принципів об'єктно-орієнтованого програмування з розділенням коду на окремі логічні модулі (файли).

Все програмне забезпечення та супровідна технічна документація повинні задовольняти ДСТУ 3008-2015 «Розробка технічної документації».

5. Стадії та етапи розробки програмного забезпечення:

- розробка алгоритмічної складової програмного забезпечення (до 15.03.2026 р.);
- об'єктно-орієнтований аналіз предметної області завдання (до 28.03.2026 р.);
- об'єктно-орієнтоване проєктування програмного забезпечення (до 30.04.2026 р.);
- розробка програмного забезпечення (до 20.04.2026 р.);
- тестування розробленого програмного забезпечення (до 30.04.2026 р.);

- демонстрація та захист програмного забезпечення (до 19.05.2026 р.).

6. Порядок контролю та приймання. Поточні результати роботи над курсовою роботою регулярно демонструються викладачу. Своєчасність виконання основних етапів графіку підготовки роботи впливає на оцінку за курсову роботу відповідно до критеріїв оцінювання.

ДОДАТОК Б

Тексти програмного коду

студентки групи ПІ-55, І курсу

Гончаренко А.С.

Обсяг програми: 12 файлів, ~1100 рядків коду.

Вид носія даних: GitHub репозиторій.

Найменування програми: Тексти програмного коду програмного забезпечення «Комп'ютерна гра «Змійка» з елементами ПІ».

Файл: *SnakeGame.csproj*

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net8.0-windows</TargetFramework>
    <Nullable>enable</Nullable>
    <UseWindowsForms>true</UseWindowsForms>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>SnakeGame</RootNamespace>
    <AssemblyName>SnakeGame</AssemblyName>
  </PropertyGroup>

</Project>
```

Файл: Program.cs

```
using System;
using System.Windows.Forms;

namespace SnakeGame
{
    /// <summary>
    /// Точка входу до програми "Змійка".
    /// Запускає головне вікно з правилами гри.
```

```

/// </summary>
internal static class Program
{
    [STAThread]
    static void Main()
    {
        ApplicationConfiguration.Initialize();
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);

        // Перед запуском основної гри показуємо вікно з правилами
        using (RulesForm rulesForm = new RulesForm())
        {
            if (rulesForm.ShowDialog() == DialogResult.OK)
            {
                Application.Run(new GameForm());
            }
        }
    }
}

```

Файл: Direction.cs

```

namespace SnakeGame
{
    /// <summary>
    /// Перелічення напрямків руху змійки.
    /// </summary>
    public enum Direction
    {
        Up,
        Down,
        Left,
        Right
    }
}

```

Файл: Point.cs

```

using System;

namespace SnakeGame
{

```

```

/// <summary>
/// Клас, що представляє координату клітинки на ігровому полі.
/// </summary>
public class Point : IEquatable<Point>
{
    public int X { get; set; }
    public int Y { get; set; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }

    /// <summary>
    /// Створює нову точку зі зміщенням у заданому напрямку.
    /// </summary>
    public Point Move(Direction dir)
    {
        return dir switch
        {
            Direction.Up => new Point(X, Y - 1),
            Direction.Down => new Point(X, Y + 1),
            Direction.Left => new Point(X - 1, Y),
            Direction.Right => new Point(X + 1, Y),
            _ => new Point(X, Y)
        };
    }

    public bool Equals(Point? other)
    {
        if (other is null) return false;
        return X == other.X && Y == other.Y;
    }

    public override bool Equals(object? obj) => Equals(obj as Point);

    public override int GetHashCode() => GetHashCode.Combine(X, Y);

    public override string ToString() => $"({X}, {Y})";
}
}

```

Файл: Field.cs

```

namespace SnakeGame
{
    /// <summary>
    /// Клас ігрового поля. Зберігає розміри та забезпечує
    /// функцію телепортації координат через краї поля.
    /// </summary>
    public class Field
    {
        public int Width { get; }
        public int Height { get; }

        public Field(int width, int height)
        {
            Width = width;
            Height = height;
        }

        /// <summary>
        /// Нормалізує координати точки відповідно до правила
        /// телепортації через стіни (тор-топология).
        /// </summary>
        public Point Wrap(Point p)
        {
            int x = ((p.X % Width) + Width) % Width;
            int y = ((p.Y % Height) + Height) % Height;
            return new Point(x, y);
        }

        /// <summary>
        /// Перевіряє чи знаходиться точка в межах поля.
        /// </summary>
        public bool Contains(Point p)
        {
            return p.X >= 0 && p.X < Width && p.Y >= 0 && p.Y < Height;
        }
    }
}

```

Файл: Snake.cs

```

using System.Collections.Generic;
using System.Drawing;

namespace SnakeGame

```

```

{
    /// <summary>
    /// Клас, що представляє змійку (керовану гравцем або ботом).
    /// Зберігає тіло, напрямок руху, життєвий стан, рахунок та колір.
    /// </summary>
    public class Snake
    {
        private List<Point> body;
        private bool growPending;

        public Direction CurrentDirection { get; private set; }
        public bool IsAlive { get; private set; } = true;
        public int Score { get; private set; } = 0;
        public Color HeadColor { get; }
        public Color BodyColor { get; }
        public string Name { get; }

        /// <summary>
        /// Голова змійки (перший елемент тіла).
        /// </summary>
        public Point Head => body[0];

        /// <summary>
        /// Все тіло змійки (тільки для читання).
        /// </summary>
        public IReadOnlyList<Point> Body => body.AsReadOnly();

        public int Length => body.Count;

        public Snake(Point startPosition, Direction startDirection, Color headColor,
            Color bodyColor, string name)
        {
            body = new List<Point> { startPosition };
            CurrentDirection = startDirection;
            HeadColor = headColor;
            BodyColor = bodyColor;
            Name = name;
        }

        /// <summary>
        /// Змінює напрямок руху, але забороняє розворот на 180°.
        /// </summary>
        public void ChangeDirection(Direction newDir)
        {
            if (IsOpposite(CurrentDirection, newDir)) return;

```

```

    CurrentDirection = newDir;
}

/// <summary>
/// Перевіряє чи два напрямки протилежні.
/// </summary>
private static bool IsOpposite(Direction a, Direction b)
{
    return (a == Direction.Up && b == Direction.Down) ||
           (a == Direction.Down && b == Direction.Up) ||
           (a == Direction.Left && b == Direction.Right) ||
           (a == Direction.Right && b == Direction.Left);
}

/// <summary>
/// Виконує крок руху змійки. Поле потрібне для телепортації через
стіни.
/// </summary>
public void Move(Field field)
{
    if (!IsAlive) return;

    Point newHead = field.Wrap(Head.Move(CurrentDirection));
    body.Insert(0, newHead);

    if (growPending)
    {
        growPending = false;
    }
    else
    {
        body.RemoveAt(body.Count - 1);
    }
}

/// <summary>
/// Позначає що змійка має вирости на наступному кроці.
/// Викликається після поглинання їжі.
/// </summary>
public void Grow(int scoreIncrement)
{
    growPending = true;
    Score += scoreIncrement;
}

```



```

    /// <summary>
    /// Переводить змійку в стан "мертва".
    /// </summary>
    public void Die()
    {
        IsAlive = false;
    }

    /// <summary>
    /// Перевіряє чи містить тіло змійки задану точку.
    /// </summary>
    public bool Contains(Point p)
    {
        foreach (var part in body)
        {
            if (part.Equals(p)) return true;
        }
        return false;
    }

    /// <summary>
    /// Перевіряє самозіткнення (голова збігається з частиною тіла).
    /// </summary>
    public bool HasSelfCollision()
    {
        for (int i = 1; i < body.Count; i++)
        {
            if (body[i].Equals(Head)) return true;
        }
        return false;
    }
}

```

Файл: Food.cs

```

using System;
using System.Collections.Generic;
using System.Drawing;

namespace SnakeGame
{
    /// <summary>
    /// Тип їжі: звичайна або бонусна.

```

```

/// </summary>
public enum FoodType
{
    Regular,
    Bonus
}

/// <summary>
/// Клас їжі на ігровому полі.
/// </summary>
public class Food
{
    public Point Position { get; }
    public FoodType Type { get; }

    /// <summary>
    /// Кількість очок, яку дає поглинання їжі.
    /// </summary>
    public int ScoreValue => Type == FoodType.Bonus ? 5 : 1;

    /// <summary>
    /// Колір їжі для відображення.
    /// </summary>
    public Color DisplayColor => Type == FoodType.Bonus ? Color.Gold :
Color.Red;

    public Food(Point position, FoodType type)
    {
        Position = position;
        Type = type;
    }

    /// <summary>
    /// Генерує нову їжу у випадковій порожній клітинці поля.
    /// </summary>
    public static Food GenerateRandom(Field field, IEnumerable<Point>
occupiedCells, Random random)
    {
        HashSet<Point> occupied = new HashSet<Point>(occupiedCells);
        List<Point> freeCells = new List<Point>();

        for (int x = 0; x < field.Width; x++)
        {
            for (int y = 0; y < field.Height; y++)
            {

```

```

        Point p = new Point(x, y);
        if (!occupied.Contains(p)) freeCells.Add(p);
    }
}

if (freeCells.Count == 0)
{
    // Поле повністю заповнене - повертаємо першу клітинку як
заглушку
    return new Food(new Point(0, 0), FoodType.Regular);
}

Point pos = freeCells[random.Next(freeCells.Count)];

// 20% шанс на бонусну їжу
FoodType type = random.Next(100) < 20 ? FoodType.Bonus :
FoodType.Regular;

    return new Food(pos, type);
}
}
}

```

Файл: PathFinder.cs

```

using System;
using System.Collections.Generic;

namespace SnakeGame
{
    /// <summary>
    /// Тип алгоритму пошуку шляху для ШІ.
    /// </summary>
    public enum PathAlgorithm
    {
        AStar,
        BFS,
        Greedy
    }

    /// <summary>
    /// Клас з реалізаціями алгоритмів пошуку найкоротшого шляху
    /// для ШІ-змійки на полі з перешкодами.
    /// </summary>

```

```

public class PathFinder
{
    /// <summary>
    /// Лічильник елементарних операцій (для оцінки практичної складності
    /// в розділі "Аналіз результатів").
    /// </summary>
    public int OperationsCount { get; private set; }

    /// <summary>
    /// Знаходить найкоротший шлях від start до target обраним методом.
    /// Повертає список точок (без стартової) або null якщо шлях не існує.
    /// </summary>
    public List<Point>? FindPath(Point start, Point target, Field field,
                                HashSet<Point> obstacles, PathAlgorithm algorithm)
    {
        OperationsCount = 0;

        return algorithm switch
        {
            PathAlgorithm.AStar => AStarSearch(start, target, field, obstacles),
            PathAlgorithm.BFS => BreadthFirstSearch(start, target, field, obstacles),
            PathAlgorithm.Greedy => GreedySearch(start, target, field, obstacles),
            _ => null
        };
    }

    /// <summary>
    /// Манхеттенська відстань з урахуванням телепортації через стіни.
    /// </summary>
    private static int ManhattanDistance(Point a, Point b, Field field)
    {
        int dx = Math.Abs(a.X - b.X);
        int dy = Math.Abs(a.Y - b.Y);
        // На торі мінімальна відстань - менша з двох
        dx = Math.Min(dx, field.Width - dx);
        dy = Math.Min(dy, field.Height - dy);
        return dx + dy;
    }

    /// <summary>
    /// Повертає сусідні клітинки з урахуванням телепортації.
    /// </summary>
    private static IEnumerable<Point> GetNeighbors(Point p, Field field)
    {
        yield return field.Wrap(p.Move(Direction.Up));
    }
}

```

```

        yield return field.Wrap(p.Move(Direction.Down));
        yield return field.Wrap(p.Move(Direction.Left));
        yield return field.Wrap(p.Move(Direction.Right));
    }

    /// <summary>
    /// Алгоритм A* (А-зірочка) - оптимальний за швидкістю та якістю.
    ///  $f(n) = g(n) + h(n)$ , де  $g$  - вартість досягнення,  $h$  - евристика.
    /// </summary>
    private List<Point>? AStarSearch(Point start, Point target, Field field,
    HashSet<Point> obstacles)
    {
        // Пріоритетна черга через SortedSet
        var openSet = new PriorityQueue<Point, int>();
        var cameFrom = new Dictionary<Point, Point>();
        var gScore = new Dictionary<Point, int>();
        var fScore = new Dictionary<Point, int>();

        gScore[start] = 0;
        fScore[start] = ManhattanDistance(start, target, field);
        openSet.Enqueue(start, fScore[start]);

        var inOpenSet = new HashSet<Point> { start };

        while (openSet.Count > 0)
        {
            OperationsCount++;
            Point current = openSet.Dequeue();
            inOpenSet.Remove(current);

            if (current.Equals(target))
            {
                return ReconstructPath(cameFrom, current);
            }

            foreach (var neighbor in GetNeighbors(current, field))
            {
                OperationsCount++;
                // Не входить в перешкоду, але дозволити цільову клітинку
                if (obstacles.Contains(neighbor) && !neighbor.Equals(target))
                    continue;

                int tentativeG = gScore[current] + 1;

                if (!gScore.ContainsKey(neighbor) || tentativeG < gScore[neighbor])

```

```

        {
            cameFrom[neighbor] = current;
            gScore[neighbor] = tentativeG;
            fScore[neighbor] = tentativeG + ManhattanDistance(neighbor,
target, field);

            if (!inOpenSet.Contains(neighbor))
            {
                openSet.Enqueue(neighbor, fScore[neighbor]);
                inOpenSet.Add(neighbor);
            }
        }
    }
}

return null; // шлях не знайдено
}

/// <summary>
/// Алгоритм BFS (пошук в ширину) - гарантує найкоротший шлях
/// у незваженому графі, але повільніший за A*.
/// </summary>
private List<Point>? BreadthFirstSearch(Point start, Point target, Field field,
HashSet<Point> obstacles)
{
    var queue = new Queue<Point>();
    var cameFrom = new Dictionary<Point, Point>();
    var visited = new HashSet<Point> { start };

    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        OperationsCount++;
        Point current = queue.Dequeue();

        if (current.Equals(target))
        {
            return ReconstructPath(cameFrom, current);
        }

        foreach (var neighbor in GetNeighbors(current, field))
        {
            OperationsCount++;
            if (visited.Contains(neighbor)) continue;

```

```

        if (obstacles.Contains(neighbor) && !neighbor.Equals(target))
continue;

        visited.Add(neighbor);
        cameFrom[neighbor] = current;
        queue.Enqueue(neighbor);
    }
}

return null;
}

/// <summary>
/// Жадібний алгоритм - на кожному кроці обирає сусіда найближчого до
цїлі.
/// Швидкий, але не гарантує знаходження шляху чи його оптимальності.
/// </summary>
private List<Point>? GreedySearch(Point start, Point target, Field field,
HashSet<Point> obstacles)
{
    var path = new List<Point>();
    var visited = new HashSet<Point> { start };
    Point current = start;

    int maxSteps = field.Width * field.Height;
    int steps = 0;

    while (!current.Equals(target) && steps < maxSteps)
    {
        OperationsCount++;
        steps++;

        Point? bestNeighbor = null;
        int bestDistance = int.MaxValue;

        foreach (var neighbor in GetNeighbors(current, field))
        {
            OperationsCount++;
            if (visited.Contains(neighbor)) continue;
            if (obstacles.Contains(neighbor) && !neighbor.Equals(target))
continue;

            int dist = ManhattanDistance(neighbor, target, field);
            if (dist < bestDistance)
            {

```

```

        bestDistance = dist;
        bestNeighbor = neighbor;
    }
}

if (bestNeighbor == null) return null; // тупик

visited.Add(bestNeighbor);
path.Add(bestNeighbor);
current = bestNeighbor;
}

return current.Equals(target) ? path : null;
}

/// <summary>
/// Відновлює шлях від cameFrom-словника.
/// </summary>
private static List<Point> ReconstructPath(Dictionary<Point, Point>
cameFrom, Point target)
{
    var path = new List<Point>();
    Point current = target;

    while (cameFrom.ContainsKey(current))
    {
        path.Add(current);
        current = cameFrom[current];
    }

    path.Reverse();
    return path;
}

/// <summary>
/// На основі першої точки шляху визначає напрямок руху.
/// </summary>
public static Direction? GetDirectionFromPath(Point head, Point next, Field
field)
{
    // Враховуємо телепортацію - перевіряємо обидва варіанти
    int dx = next.X - head.X;
    int dy = next.Y - head.Y;

    // Нормалізація для тор-поля

```



```

        if (dx > field.Width / 2) dx -= field.Width;
        if (dx < -field.Width / 2) dx += field.Width;
        if (dy > field.Height / 2) dy -= field.Height;
        if (dy < -field.Height / 2) dy += field.Height;

        if (dx == 1) return Direction.Right;
        if (dx == -1) return Direction.Left;
        if (dy == 1) return Direction.Down;
        if (dy == -1) return Direction.Up;

        return null;
    }
}

```

Файл: AIController.cs

```

using System.Collections.Generic;

namespace SnakeGame
{
    /// <summary>
    /// Клас, що керує змією-ботом. На кожному кроці виконує
    /// пошук шляху до найближчої їжі та обирає напрямок руху.
    /// </summary>
    public class AIController
    {
        private readonly Snake snake;
        private readonly Field field;
        private readonly Pathfinder pathFinder;

        public PathAlgorithm Algorithm { get; set; }

        /// <summary>
        /// Загальний лічильник елементарних операцій ШІ
        /// (накопичується між викликами для аналізу ефективності).
        /// </summary>
        public long TotalOperations { get; private set; }

        /// <summary>
        /// Кількість викликів пошуку шляху.
        /// </summary>
        public int PathFindCallsCount { get; private set; }
    }
}

```

```

    public AIController(Snake snake, Field field, PathAlgorithm algorithm =
PathAlgorithm.AStar)
    {
        this.snake = snake;
        this.field = field;
        this.pathFinder = new Pathfinder();
        this.Algorithm = algorithm;
    }

    /// <summary>
    /// Виконує крок прийняття рішення. Знаходить найближчу їжу,
    /// будує шлях до неї та обирає наступний напрямок.
    /// </summary>
    public void MakeDecision(IEnumerable<Food> foods, IEnumerable<Snake>
allSnakes)
    {
        if (!snake.IsAlive) return;

        // Збираємо перешкоди: тіла всіх змійок крім хвоста (хвіст рухається)
        HashSet<Point> obstacles = BuildObstacleSet(allSnakes);

        // Шукаємо найближчу їжу за манхеттенською відстанню
        Food? targetFood = FindClosestFood(foods);
        if (targetFood == null) return;

        // Шукаємо шлях обраним алгоритмом
        List<Point>? path = pathFinder.FindPath(
            snake.Head, targetFood.Position, field, obstacles, Algorithm);

        TotalOperations += pathFinder.OperationsCount;
        PathFindCallsCount++;

        if (path == null || path.Count == 0)
        {
            // Шлях не знайдено - намагаємось хоча б рухатись у безпечну
клітинку
            TryMoveSafely(obstacles);
            return;
        }

        // Визначаємо напрямок до першої точки шляху
        Direction? dir = PathFinder.GetDirectionFromPath(snake.Head, path[0],
field);
        if (dir.HasSelfCollisionRisk(snake) == false && dir.HasValue)
        {

```

```

        snake.ChangeDirection(dir.Value);
    }
}

/// <summary>
/// Будує множину перешкод для пошуку шляху.
/// </summary>
private HashSet<Point> BuildObstacleSet(IEnumerable<Snake> allSnakes)
{
    var obstacles = new HashSet<Point>();
    foreach (var s in allSnakes)
    {
        if (!s.IsAlive) continue;
        int idx = 0;
        foreach (var part in s.Body)
        {
            // Хвіст пропускаємо - він зрушиться, тож наступним кроком
            клітинка буде вільна
            bool isOwnTail = (s == snake) && (idx == s.Length - 1);
            if (!isOwnTail)
            {
                obstacles.Add(part);
            }
            idx++;
        }
    }
    return obstacles;
}

/// <summary>
/// Знаходить їжу, найближчу до голови змійки.
/// </summary>
private Food? FindClosestFood(IEnumerable<Food> foods)
{
    Food? best = null;
    int bestDist = int.MaxValue;

    foreach (var f in foods)
    {
        int dx = System.Math.Abs(snake.Head.X - f.Position.X);
        int dy = System.Math.Abs(snake.Head.Y - f.Position.Y);
        dx = System.Math.Min(dx, field.Width - dx);
        dy = System.Math.Min(dy, field.Height - dy);
        int dist = dx + dy;
    }
}

```

```

        if (dist < bestDist)
        {
            bestDist = dist;
            best = f;
        }
    }

    return best;
}

/// <summary>
/// Резервна стратегія: коли шлях не знайдено, рухаємось у будь-яку
/// безпечну сусідню клітинку, щоб не врізатись.
/// </summary>
private void TryMoveSafely(HashSet<Point> obstacles)
{
    Direction[] dirs = { Direction.Up, Direction.Down, Direction.Left,
Direction.Right };
    foreach (var d in dirs)
    {
        Point next = field.Wrap(snake.Head.Move(d));
        if (!obstacles.Contains(next))
        {
            snake.ChangeDirection(d);
            return;
        }
    }
}

/// <summary>
/// Розширення для Direction (допоміжна перевірка).
/// </summary>
internal static class DirectionExtensions
{
    public static bool HasSelfCollisionRisk(this Direction? dir, Snake snake)
    {
        // Заглушка для семантики - ChangeDirection сам захищає від
розвороту
        return false;
    }
}
}

```

Файл: GameState.cs

```
using System.Collections.Generic;
using System.Drawing;
using System;

namespace SnakeGame
{
    /// <summary>
    /// Стан гри.
    /// </summary>
    public enum GameStatus
    {
        Running,
        PlayerWon,
        PlayerLost,
        Draw
    }

    /// <summary>
    /// Центральний клас, що інкапсулює стан гри: поле, змійки, їжу,
    /// логіку оновлення та обробку зіткнень.
    /// </summary>
    public class GameState
    {
        public Field Field { get; }
        public Snake Player { get; }
        public Snake Bot { get; }
        public AIController AI { get; }
        public List<Food> Foods { get; private set; }
        public GameStatus Status { get; private set; }

        private readonly Random random;
        private const int FoodCount = 3;

        public GameState(int width, int height, PathAlgorithm aiAlgorithm =
PathAlgorithm.AStar)
        {
            Field = new Field(width, height);
            random = new Random();

            // Гравець стартує зліва, бот - справа
            Player = new Snake(
                new Point(width / 4, height / 2),
                Direction.Right,
```

```

        Color.FromArgb(46, 204, 113),    // яскраво-зелений
        Color.FromArgb(39, 174, 96),    // темно-зелений
        "Гравець"
    );

    Bot = new Snake(
        new Point(3 * width / 4, height / 2),
        Direction.Left,
        Color.FromArgb(231, 76, 60),    // яскраво-червоний
        Color.FromArgb(192, 57, 43),    // темно-червоний
        "Бот (III)"
    );

    AI = new AIController(Bot, Field, aiAlgorithm);
    Foods = new List<Food>();
    Status = GameStatus.Running;

    // Початкова генерація їжі
    for (int i = 0; i < FoodCount; i++)
    {
        AddFood();
    }
}

/// <summary>
/// Виконує один такт гри: III приймає рішення, обидві змійки
рухаються,
/// перевіряються зіткнення та поглинання їжі.
/// </summary>
public void Update()
{
    if (Status != GameStatus.Running) return;

    // Спочатку III приймає рішення на основі поточного стану
    AI.MakeDecision(Foods, new[] { Player, Bot });

    // Рухаємо обидві змійки одночасно
    Player.Move(Field);
    Bot.Move(Field);

    // Обробляємо поглинання їжі
    HandleFoodConsumption(Player);
    HandleFoodConsumption(Bot);

    // Перевіряємо зіткнення та визначаємо стан гри

```

```

    CheckCollisions();
    UpdateGameStatus();
}

/// <summary>
/// Перевіряє чи з'їла змійка їжу. Якщо так - їжа зникає, з'являється нова.
/// </summary>
private void HandleFoodConsumption(Snake snake)
{
    if (!snake.IsAlive) return;

    for (int i = Foods.Count - 1; i >= 0; i--)
    {
        if (Foods[i].Position.Equals(snake.Head))
        {
            snake.Grow(Foods[i].ScoreValue);
            Foods.RemoveAt(i);
            AddFood();
        }
    }
}

/// <summary>
/// Додає нову їжу у випадкову порожню клітинку.
/// </summary>
private void AddFood()
{
    var occupied = new List<Point>();
    occupied.AddRange(Player.Body);
    occupied.AddRange(Bot.Body);
    foreach (var f in Foods) occupied.Add(f.Position);

    Foods.Add(Food.GenerateRandom(Field, occupied, random));
}

/// <summary>
/// Перевіряє всі типи зіткнень: самозіткнення та зіткнення з іншою
змією.
/// </summary>
private void CheckCollisions()
{
    // Самозіткнення
    if (Player.HasSelfCollision()) Player.Die();
    if (Bot.HasSelfCollision()) Bot.Die();
}

```

```

// Зіткнення головою у тіло іншої змійки
if (Player.IsAlive && Bot.IsAlive)
{
    // Голова в голову - обоє програли
    if (Player.Head.Equals(Bot.Head))
    {
        Player.Die();
        Bot.Die();
        return;
    }

    if (Bot.Contains(Player.Head)) Player.Die();
    if (Player.Contains(Bot.Head)) Bot.Die();
}

}

/// <summary>
/// Оновлює загальний статус гри на основі стану змійок.
/// </summary>
private void UpdateGameStatus()
{
    if (Player.IsAlive && Bot.IsAlive)
    {
        Status = GameStatus.Running;
    }
    else if (!Player.IsAlive && !Bot.IsAlive)
    {
        // Обидві мертві - визначаємо за рахунком
        if (Player.Score > Bot.Score) Status = GameStatus.PlayerWon;
        else if (Bot.Score > Player.Score) Status = GameStatus.PlayerLost;
        else Status = GameStatus.Draw;
    }
    else if (!Player.IsAlive)
    {
        Status = GameStatus.PlayerLost;
    }
    else // !Bot.IsAlive
    {
        Status = GameStatus.PlayerWon;
    }
}
}
}
}

```


Файл: RulesForm.cs

```
using System.Drawing;
using System.Windows.Forms;
// Alias щоб розрізнити System.Drawing.Point та SnakeGame.Point
using UIPoint = System.Drawing.Point;

namespace SnakeGame
{
    /// <summary>
    /// Форма-вітання з правилами гри. Відкривається першою при запуску
    програми.
    /// </summary>
    public class RulesForm : Form
    {
        private Button startButton = null!;
        private Button exitButton = null!;
        private Label titleLabel = null!;
        private Label rulesLabel = null!;

        public RulesForm()
        {
            InitializeComponent();
        }

        private void InitializeComponent()
        {
            // Налаштування форми
            Text = "Змійка - Правила гри";
            Size = new Size(620, 580);
            StartPosition = FormStartPosition.CenterScreen;
            FormBorderStyle = FormBorderStyle.FixedDialog;
            MaximizeBox = false;
            BackColor = Color.FromArgb(30, 30, 40);

            // Заголовок
            titleLabel = new Label
            {
                Text = "ЗМІЙКА",
                Font = new Font("Segoe UI", 28, FontStyle.Bold),
                ForeColor = Color.FromArgb(46, 204, 113),
                Location = new UIPoint(0, 20),
                Size = new Size(620, 50),
                TextAlign = ContentAlignment.MiddleCenter
            };
        }
    }
}
```

```

Controls.Add(titleLabel);

// Текст правил
rulesLabel = new Label
{
    Text = "ПРАВИЛА ГРИ:\r\n\r\n" +
        " • Керуйте зеленою змією за допомогою клавіш-стрілок\r\n"
+
        " (↑ ↓ ← →) або клавіш WASD.\r\n\r\n" +
        " • Збирайте червону їжу — за неї дається 1 очко.\r\n\r\n" +
        " • Шукайте золоту бонусну їжу — вона дає 5 очок!\r\n\r\n" +
        " • Червона змійка-бот керується штучним інтелектом\r\n" +
        " (алгоритм A*). Вона також полює за їжею.\r\n\r\n" +
        " • При зіткненні зі стіною змійка з'являється з протилежного
боку.\r\n\r\n" +
        " • НЕ ВРІЗАЙТЕСЬ у власне тіло чи у тіло змійки-
суперника!\r\n\r\n" +
        " • Перемагає той, хто залишиться живим. У разі загибелі обох
—\r\n" +
        " хто набрав більше очок.\r\n\r\n" +
        " • Пауза/поновлення гри — клавіша SPACE.",
    Font = new Font("Segoe UI", 11, FontStyle.Regular),
    ForeColor = Color.White,
    Location = new UIPoint(40, 80),
    Size = new Size(540, 360),
    TextAlign = ContentAlignment.TopLeft
};
Controls.Add(rulesLabel);

// Кнопка "Почати гру"
startButton = new Button
{
    Text = "ПОЧАТИ ГРУ",
    Font = new Font("Segoe UI", 12, FontStyle.Bold),
    Size = new Size(200, 50),
    Location = new UIPoint(80, 470),
    BackColor = Color.FromArgb(46, 204, 113),
    ForeColor = Color.White,
    FlatStyle = FlatStyle.Flat,
    DialogResult = DialogResult.OK
};
startButton.FlatAppearance.BorderSize = 0;
Controls.Add(startButton);

// Кнопка "Вихід"

```

```

        exitButton = new Button
        {
            Text = "ВИХІД",
            Font = new Font("Segoe UI", 12, FontStyle.Bold),
            Size = new Size(200, 50),
            Location = new UIPoint(340, 470),
            BackColor = Color.FromArgb(231, 76, 60),
            ForeColor = Color.White,
            FlatStyle = FlatStyle.Flat,
            DialogResult = DialogResult.Cancel
        };
        exitButton.FlatAppearance.BorderSize = 0;
        Controls.Add(exitButton);

        AcceptButton = startButton;
        CancelButton = exitButton;
    }
}

```

Файл: GameForm.cs

```

using System;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;
// Alias щоб розрізняти System.Drawing.Point (для UI-координат)
// та SnakeGame.Point (для координат на ігровому полі)
using UIPoint = System.Drawing.Point;

namespace SnakeGame
{
    /// <summary>
    /// Головна форма гри. Відповідає за рендеринг ігрового стану,
    /// обробку клавіатурного вводу та таймер ігрових тактів.
    /// </summary>
    public class GameForm : Form
    {
        private const int FieldWidth = 20;
        private const int FieldHeight = 20;
        private const int CellSize = 28;
        private const int InfoPanelWidth = 240;
        private const int GameTickMs = 130;
    }
}

```

```

private GameState gameState;
private System.Windows.Forms.Timer gameTimer = null!;
private Panel gamePanel = null!;
private Panel infoPanel = null!;
private Label playerScoreLabel = null!;
private Label botScoreLabel = null!;
private Label statusLabel = null!;
private Label aiOpsLabel = null!;
private Button restartButton = null!;
private bool isPaused;

public GameForm()
{
    gameState = new GameState(FieldWidth, FieldHeight);
    InitializeComponent();
    StartGame();
}

private void InitializeComponent()
{
    Text = "Змійка - комп'ютерна гра з елементами III";
    Size = new Size(FieldWidth * CellSize + InfoPanelWidth + 40,
        FieldHeight * CellSize + 60);
    StartPosition = FormStartPosition.CenterScreen;
    FormBorderStyle = FormBorderStyle.FixedSingle;
    MaximizeBox = false;
    BackColor = Color.FromArgb(30, 30, 40);
    KeyPreview = true;
    DoubleBuffered = true;

    // Панель ігрового поля
    gamePanel = new Panel
    {
        Location = new UIPoint(10, 10),
        Size = new Size(FieldWidth * CellSize, FieldHeight * CellSize),
        BackColor = Color.FromArgb(20, 20, 30)
    };
    // Ввімкнути подвійну буферизацію через рефлексію
    typeof(Control).GetProperty("DoubleBuffered",
        System.Reflection.BindingFlags.Instance |
        System.Reflection.BindingFlags.NonPublic)?
        .SetValue(gamePanel, true, null);
    gamePanel.Paint += GamePanel_Paint;
    Controls.Add(gamePanel);
}

```

```

// Інформаційна панель
infoPanel = new Panel
{
    Location = new UIPoint(FieldWidth * CellSize + 20, 10),
    Size = new Size(InfoPanelWidth, FieldHeight * CellSize),
    BackColor = Color.FromArgb(40, 40, 55)
};
Controls.Add(infoPanel);

BuildInfoPanel();

// Таймер гри
gameTimer = new System.Windows.Forms.Timer
{
    Interval = GameTickMs
};
gameTimer.Tick += GameTimer_Tick;
}

private void BuildInfoPanel()
{
    var titleLabel = new Label
    {
        Text = "СТАТУС ГРИ",
        Font = new Font("Segoe UI", 13, FontStyle.Bold),
        ForeColor = Color.White,
        Location = new UIPoint(10, 15),
        Size = new Size(220, 30),
        TextAlign = ContentAlignment.MiddleCenter
    };
    infoPanel.Controls.Add(titleLabel);

    playerScoreLabel = new Label
    {
        Text = "Гравець: 0",
        Font = new Font("Segoe UI", 12, FontStyle.Bold),
        ForeColor = Color.FromArgb(46, 204, 113),
        Location = new UIPoint(15, 70),
        Size = new Size(210, 30)
    };
    infoPanel.Controls.Add(playerScoreLabel);

    botScoreLabel = new Label
    {

```

```

        Text = "Бот (III): 0",
        Font = new Font("Segoe UI", 12, FontStyle.Bold),
        ForeColor = Color.FromArgb(231, 76, 60),
        Location = new UIPoint(15, 105),
        Size = new Size(210, 30)
    };
    infoPanel.Controls.Add(botScoreLabel);

    statusLabel = new Label
    {
        Text = "Гра триває",
        Font = new Font("Segoe UI", 11, FontStyle.Italic),
        ForeColor = Color.LightGray,
        Location = new UIPoint(15, 160),
        Size = new Size(210, 60),
        TextAlign = ContentAlignment.MiddleLeft
    };
    infoPanel.Controls.Add(statusLabel);

    aiOpsLabel = new Label
    {
        Text = "Операцій III: 0\r\nВикликів пошуку: 0",
        Font = new Font("Segoe UI", 9, FontStyle.Regular),
        ForeColor = Color.LightGray,
        Location = new UIPoint(15, 240),
        Size = new Size(210, 60)
    };
    infoPanel.Controls.Add(aiOpsLabel);

    var hintLabel = new Label
    {
        Text = "↑↓←→ або WASD\r\nSPACE - пауза",
        Font = new Font("Segoe UI", 9, FontStyle.Regular),
        ForeColor = Color.Gray,
        Location = new UIPoint(15, 320),
        Size = new Size(210, 50)
    };
    infoPanel.Controls.Add(hintLabel);

    restartButton = new Button
    {
        Text = "РЕСТАРТ",
        Font = new Font("Segoe UI", 11, FontStyle.Bold),
        Size = new Size(200, 45),
        Location = new UIPoint(20, 410),

```

```

        BackColor = Color.FromArgb(52, 152, 219),
        ForeColor = Color.White,
        FlatStyle = FlatStyle.Flat
    };
    restartButton.FlatAppearance.BorderSize = 0;
    restartButton.Click += RestartButton_Click;
    infoPanel.Controls.Add(restartButton);

    var saveButton = new Button
    {
        Text = "ЗБЕРЕГТИ РЕЗУЛЬТАТ",
        Font = new Font("Segoe UI", 9, FontStyle.Bold),
        Size = new Size(200, 35),
        Location = new UIPoint(20, 465),
        BackColor = Color.FromArgb(155, 89, 182),
        ForeColor = Color.White,
        FlatStyle = FlatStyle.Flat
    };
    saveButton.FlatAppearance.BorderSize = 0;
    saveButton.Click += SaveButton_Click;
    infoPanel.Controls.Add(saveButton);
}

private void StartGame()
{
    gameTimer.Start();
    isPaused = false;
    Focus();
}

private void GameTimer_Tick(object? sender, EventArgs e)
{
    if (gameState.Status != GameStatus.Running)
    {
        gameTimer.Stop();
        ShowGameOverDialog();
        return;
    }

    gameState.Update();
    UpdateInfoPanel();
    gamePanel.Invalidate();
}

private void UpdateInfoPanel()

```

```

    {
        playerScoreLabel.Text = $"Гравець: {gameState.Player.Score} (довж.
{gameState.Player.Length})";
        botScoreLabel.Text = $"Бот (III): {gameState.Bot.Score} (довж.
{gameState.Bot.Length})";

        string st = gameState.Status switch
        {
            GameStatus.Running => isPaused ? "ПАУЗА" : "Гра триває",
            GameStatus.PlayerWon => "ВИ ПЕРЕМОГЛИ!",
            GameStatus.PlayerLost => "Ви програли...",
            GameStatus.Draw => "Нічия",
            _ => ""
        };
        statusLabel.Text = st;

        aiOpsLabel.Text = $"Операцій III: {gameState.AI.TotalOperations}\r\n"
+
        $"Викликів пошуку: {gameState.AI.PathFindCallsCount}";
    }

private void GamePanel_Paint(object? sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.SmoothingMode = SmoothingMode.AntiAlias;

    // Малюємо сітку поля
    DrawGrid(g);

    // Малюємо їжу
    foreach (var food in gameState.Foods)
    {
        DrawFood(g, food);
    }

    // Малюємо змійки
    DrawSnake(g, gameState.Bot);
    DrawSnake(g, gameState.Player);
}

private void DrawGrid(Graphics g)
{
    using var pen = new Pen(Color.FromArgb(40, 40, 60), 1);
    for (int x = 0; x <= FieldWidth; x++)
    {

```



```

        g.DrawLine(pen, x * CellSize, 0, x * CellSize, FieldHeight * CellSize);
    }
    for (int y = 0; y <= FieldHeight; y++)
    {
        g.DrawLine(pen, 0, y * CellSize, FieldWidth * CellSize, y * CellSize);
    }
}

private void DrawFood(Graphics g, Food food)
{
    int padding = food.Type == FoodType.Bonus ? 2 : 4;
    var rect = new Rectangle(
        food.Position.X * CellSize + padding,
        food.Position.Y * CellSize + padding,
        CellSize - 2 * padding,
        CellSize - 2 * padding);

    using var brush = new SolidBrush(food.DisplayColor);
    g.FillEllipse(brush, rect);

    if (food.Type == FoodType.Bonus)
    {
        using var pen = new Pen(Color.Yellow, 2);
        g.DrawEllipse(pen, rect);
    }
}

private void DrawSnake(Graphics g, Snake snake)
{
    if (!snake.IsAlive)
    {
        // Мертва змійка - сіра
        DrawSnakeBody(g, snake, Color.DimGray, Color.Gray);
        return;
    }
    DrawSnakeBody(g, snake, snake.HeadColor, snake.BodyColor);
}

private void DrawSnakeBody(Graphics g, Snake snake, Color headColor,
Color bodyColor)
{
    int idx = 0;
    foreach (var part in snake.Body)
    {
        int padding = 2;

```

```

var rect = new Rectangle(
    part.X * CellSize + padding,
    part.Y * CellSize + padding,
    CellSize - 2 * padding,
    CellSize - 2 * padding);

Color c = idx == 0 ? headColor : bodyColor;
using var brush = new SolidBrush(c);
g.FillRectangle(brush, rect);

// Голова - 3 очима
if (idx == 0)
{
    DrawSnakeEyes(g, snake, rect);
}
idx++;
}
}

private void DrawSnakeEyes(Graphics g, Snake snake, Rectangle headRect)
{
    using var eyeBrush = new SolidBrush(Color.White);
    using var pupilBrush = new SolidBrush(Color.Black);
    int eyeSize = 5;
    int pupilSize = 3;

    // Розташування очей залежно від напрямку
    int leftX, leftY, rightX, rightY;
    switch (snake.CurrentDirection)
    {
        case Direction.Up:
            leftX = headRect.X + 4; leftY = headRect.Y + 4;
            rightX = headRect.Right - 4 - eyeSize; rightY = headRect.Y + 4;
            break;
        case Direction.Down:
            leftX = headRect.X + 4; leftY = headRect.Bottom - 4 - eyeSize;
            rightX = headRect.Right - 4 - eyeSize; rightY = headRect.Bottom - 4 -
eyeSize;
            break;
        case Direction.Left:
            leftX = headRect.X + 4; leftY = headRect.Y + 4;
            rightX = headRect.X + 4; rightY = headRect.Bottom - 4 - eyeSize;
            break;
        default: // Right
            leftX = headRect.Right - 4 - eyeSize; leftY = headRect.Y + 4;

```

```

        rightX = headRect.Right - 4 - eyeSize; rightY = headRect.Bottom - 4 -
eyeSize;
        break;
    }

    g.FillEllipse(eyeBrush, leftX, leftY, eyeSize, eyeSize);
    g.FillEllipse(eyeBrush, rightX, rightY, eyeSize, eyeSize);
    g.FillEllipse(pupilBrush, leftX + 1, leftY + 1, pupilSize, pupilSize);
    g.FillEllipse(pupilBrush, rightX + 1, rightY + 1, pupilSize, pupilSize);
}

protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
{
    switch (keyData)
    {
        case Keys.Up:
        case Keys.W:
            // Додано перевірку: дія виконується ТІЛЬКИ якщо гра НЕ на
паузі
            if (!isPaused) gameState.Player.ChangeDirection(Direction.Up);
            return true;

        case Keys.Down:
        case Keys.S:
            if (!isPaused) gameState.Player.ChangeDirection(Direction.Down);
            return true;

        case Keys.Left:
        case Keys.A:
            if (!isPaused) gameState.Player.ChangeDirection(Direction.Left);
            return true;

        case Keys.Right:
        case Keys.D:
            if (!isPaused) gameState.Player.ChangeDirection(Direction.Right);
            return true;

        case Keys.Space:
            // Пробіл залишаємо без змін, щоб можна було зняти з паузи
            TogglePause();
            return true;
    }

    // Якщо натиснута інша клавіша, викликаємо базовий метод
    return base.ProcessCmdKey(ref msg, keyData);
}

```

```

    }
    private void TogglePause()
    {
        if (gameState.Status != GameStatus.Running) return;

        if (isPaused)
        {
            gameTimer.Start();
            isPaused = false;
        }
        else
        {
            gameTimer.Stop();
            isPaused = true;
        }
        UpdateInfoPanel();
    }

    private void RestartButton_Click(object? sender, EventArgs e)
    {
        gameTimer.Stop();
        gameState = new GameState(FieldWidth, FieldHeight);
        UpdateInfoPanel();
        gamePanel.Invalidate();
        StartGame();
    }

    private void ShowGameOverDialog()
    {
        string msg = gameState.Status switch
        {
            GameStatus.PlayerWon => $"Вітаємо! Ви перемогли!\r\nВаш
рахунок: {gameState.Player.Score}",
            GameStatus.PlayerLost => $"На жаль, ви програли.\r\nВаш рахунок:
{gameState.Player.Score}\r\nРахунок бота: {gameState.Bot.Score}",
            GameStatus.Draw => $"Нічия! Обидві змійки загинули з однаковим
рахунком ({gameState.Player.Score}).",
            _ => "Гра завершена."
        };
        MessageBox.Show(msg, "Гра завершена", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
    }

    private void SaveButton_Click(object? sender, EventArgs e)
    {

```

```

try
{
    using var dialog = new SaveFileDialog
    {
        Filter = "Текстовий файл (*.txt)|*.txt",
        FileName =
$"snake_result_{DateTime.Now:yyyyMMdd_HH:mm:ss}.txt"
    };
    if (dialog.ShowDialog() == DialogResult.OK)
    {
        string content = $"Результати гри \"Змійка\" \r\n" +
            $"Дата: {DateTime.Now:yyyy-MM-dd HH:mm:ss} \r\n" +
            $"----- \r\n" +
            $"Гравець: {gameState.Player.Score} очок (довжина
{gameState.Player.Length}) \r\n" +
            $"Бот (III): {gameState.Bot.Score} очок (довжина
{gameState.Bot.Length}) \r\n" +
            $"Алгоритм III: {gameState.AI.Algorithm} \r\n" +
            $"Загалом операцій III:
{gameState.AI.TotalOperations} \r\n" +
            $"Кількість викликів пошуку:
{gameState.AI.PathFindCallsCount} \r\n" +
            $"Статус: {gameState.Status} \r\n";
        System.IO.File.WriteAllText(dialog.FileName, content);
        MessageBox.Show("Результати збережено!", "Успіх",
            MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}
catch (Exception ex)
{
    MessageBox.Show($"Помилка збереження: {ex.Message}",
"Помилка",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
}
}

```